

TP 1 : Simulation d'ordonnancement

2SN ASR

2019-2020

Objectifs :

- Comprendre un algorithme d'ordonnancement :
 - de type tourniquet ;
 - par gestion de priorités.
- Gestion de tâches périodiques dans un contexte temps réel

1 Pour commencer

L'outil STORM est un outil de simulation d'ordonnancement temps réel multiprocesseurs. A partir d'une spécification des tâches temps réel et du choix de la politique d'ordonnancement, l'outil simule l'exécution de ces tâches. La spécification est décrite dans un fichier `.xml`.

L'archive `TP_ORDO.tar` contient les fichiers suivants :

1. la manuel d'utilisation de STORM `User-guide-V3.2.pdf`
2. la manuel du concepteur `Designer-guideV3.3_version1.pdf`
3. l'exécutable `storm-3-2.jar`
4. un exemple d'ordonnanceur FIFO `my_FIFO_Scheduler.java` et la spécification d'une exécution `Test-FIFO.xml`
5. un fichier `my_RR_Scheduler.java` à compléter pour construire un ordonnancement round-robin, avec sa spécification `Test-RR.xml`
6. un fichier `my_RM_Scheduler.java` à compléter pour construire un ordonnancement rate monotonic, avec sa spécification `Test-RMA.xml`

Les fichiers `my_FIFO_Scheduler.java`, `my_RR_Scheduler.java` et `my_RM_Scheduler.java` sont pour le moment identiques, nous les ferons évoluer par la suite.

Différents types d'ordonnancement sont accessibles via le fichier `.xml` mais nous allons ici utiliser des ordonnancements personnalisés écrits en Java. Pour tester un ordonnancement, il faut effectuer les opérations suivantes :

1. compiler le code : `javac -cp ./storm-3-2.jar my_FIFO_Scheduler.java`
2. lancer l'outil : `java -cp ../storm-3-2.jar programme.programme`
3. exécution de la spécification : `exec ./Test-FIFO.xml`
4. affichage de l'exécution : `pa` (plotall)

2 Un premier type d'ordonnancement

Dans cet exercice, nous allons étudier le programme `my_FIFO_Scheduler.java`.

Exercice 1 : Etude d'un algorithme d'ordonnancement FIFO

Après avoir téléchargé et décompressé l'archive, compilez le programme `my_FIFO_Scheduler.java` et exécutez le. Que constatez-vous ?

Exercice 2 : Prise en compte de la durée d'exécution

Modifiez le code de `my_FIFO_Scheduler.java` de manière à tenir compte de la durée d'exécution des tâches

3 Ordonnancement de type tourniquet (Round-Robin)

L'ordonnancement de type tourniquet (ou Round-Robin) utilise un quantum de temps pendant lequel une tâche peut s'exécuter sur le processeur :

- les tâches sont stockées dans une file d'attente de type FIFO ;
- une tâche en début de file peut s'exécuter sur le processeur pendant la durée du quantum ;
- à l'expiration du quantum, la tâche qui s'exécute est préemptée et est insérée à la fin de la file d'attente (si elle n'a pas terminée son exécution), une nouvelle tâche peut s'exécuter.

Exercice 3 : Impact du choix du quantum de temps

Doit-on choisir une durée longue ou courte pour le quantum de temps ? Pourquoi ?

Exercice 4 : Simulation du Round-Robin

Modifiez le programme `my_RR_Scheduler.java` afin de simuler le comportement du Round-Robin.

4 Ordonnancement temps réel à priorité fixe

Une tâche périodique est caractérisée par les paramètres suivants, illustré par la figure 1 :

- sa date de première occurrence T_0 ;
- sa période P ;
- sa durée (maximum) d'exécution C .

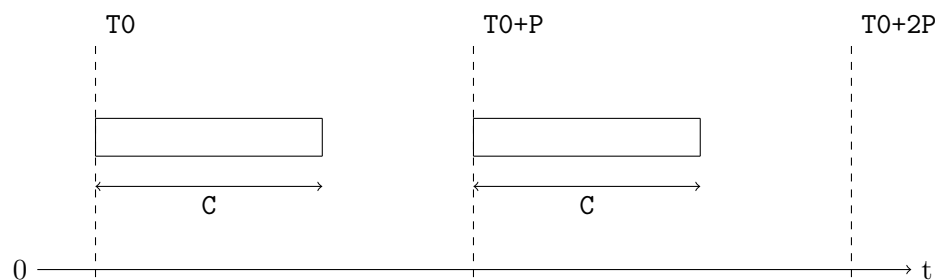


Figure 1 – Une tâche périodique

Le but d'un algorithme d'ordonnancement est alors de définir un séquençage permettant à un ensemble de tâches de s'exécuter sur un processeur.

Le RMA (*Rate Monotonic Algorithm*) est un algorithme attribuant à chaque tâche une priorité qui décroît avec sa période puis ordonnant les tâches en fonction de leur priorité.

Exercice 5 : Simulation du RMA

Modifiez le programme `my_RM_Scheduler.java` permettant de simuler le comportement du RMA.